

1 Attack Strategy

1.1 Correlation Power Analysis (CPA) — Overview

Our goal is to recover each secret coefficient $s[j]$ independently by exploiting data-dependent power leakage. We assume the device power at time sample t satisfies

$$P_i(t) \approx \alpha \cdot \text{HW}(v(k, \mathbf{r}_i)) + \beta + \varepsilon_i,$$

where $v(k, \mathbf{r}_i)$ is a hypothesised intermediate value, ε_i is measurement noise, and the correct key maximises the Pearson correlation

$$\hat{s}[j] = \arg \max_{k \in \mathbb{Z}_q} \max_t \left| \frac{\sum_i (\text{HW}_i^{(k)} - \overline{\text{HW}}^{(k)}) (P_i(t) - \overline{P}(t))}{N \sigma(\text{HW}^{(k)}) \sigma(P(\cdot, t))} \right|.$$

1.2 Leakage Model

Four candidate leakage models were tested for coefficient $j = 0$ and compared by their signal-to-noise ratio (true-key correlation divided by wrong-key correlation):

Model	True-key corr.	Wrong-key corr.
$\text{HW}(r[j] \cdot k \bmod q)$	0.355	0.064
$\text{HW}((r[j] \cdot k) \bmod 2^{16})$	0.051	0.060
$\text{HW}((r[j] \cdot k \cdot Q_{\text{INV}}) \bmod 2^{16})$	0.165	0.062
$\text{HW}(\text{fqmul}(k, r[j]))$	0.109	0.068

The model $\text{HW}(r[j] \cdot k \bmod q)$ achieves the highest signal-to-noise ratio ($\approx 5.5\times$) and is used throughout. This indicates that dominant leakage originates from the modular multiplication step, before the Montgomery reduction is applied.

1.3 Auto-Detection of Leakage Time Samples

Rather than hard-coding a timing formula, we *detect* the leaking time sample for each coefficient automatically using a sparse-projection technique. For each coefficient position c_i we build a $(q \times T)$ projection matrix

$$W[c_i]_{r,t} = \sum_{\substack{n=1 \\ \text{polys}[n, c_i]=r}}^N \tilde{P}_n(t),$$

where $\tilde{P}_n(t)$ is the column-normalised trace. This groups traces by the polynomial value that appeared at position c_i and sums them; the column whose 2-norm is largest is the sample where power most strongly depends on the coefficient value:

$$\text{peak}(c_i) = \arg \max_t \|W[c_i]_{:,t}\|_2.$$

Why this works. At the true leaking time t^* , traces with the same polynomial input r experience similar power (because the device is computing the same product $r \cdot s[j]$).

Summing them coherently amplifies the signal while random noise from other traces cancels. At non-leaking samples, grouping by r is uncorrelated with power, so all column norms are comparable noise. No knowledge of the secret or timing formula is required.

On the provided dataset the auto-detected peaks agree with the analytic formula $t(j) = 5\lfloor j/2 \rfloor + 1 + (j \bmod 2)$ for coefficients $j = 0\text{--}127$, and diverge slightly for $j = 128\text{--}255$ (tied column norms from a different loop-order in that half of the NTT). The shift sweep described in Section 1.4 compensates for this.

1.4 Lookup-Table CPA with Shift Sweep

The main computational bottleneck is evaluating $\text{HW}(r[j] \cdot k \bmod q)$ for all N traces, all $q = 3329$ key hypotheses, and all 256 coefficients. We avoid the naïve $O(C \times N \times q)$ multiplication by precomputing a $(C \times q)$ summary matrix

$$\text{coeff_tr}[c_i, r] = \sum_{\substack{n=1 \\ \text{polys}[n, c_i]=r}}^N \tilde{P}_n(\text{peak}(c_i) + \delta),$$

for each time shift $\delta \in \{-5, \dots, +5\}$. For a fixed hypothesis s , the CPA score for all 256 coefficients simultaneously is

$$\text{score}[s] = \|\text{coeff_tr} \mathbf{h}_s\| \in \mathbb{R}^{256}, \quad (\mathbf{h}_s)_r = \text{HW}(r \cdot s \bmod q).$$

This reduces the inner loop to a single $(256 \times q) \cdot (q,)$ matrix-vector product per hypothesis, giving a total of $3329 \times 11 \times 256 \approx 9.4 \times 10^6$ floating-point operations — completing in ≈ 8 s.

The 11-shift sweep ensures that even if the auto-detected peak is off by a few samples (due to noise or tied column norms), the correct leaking sample is still included.

1.5 Montgomery Correction

Because Kyber uses Montgomery arithmetic internally, the raw CPA winner is a scaled value:

$$k_{\text{raw}} \equiv s[j] \cdot 2704 \pmod{q}.$$

We recover the true coefficient by

$$s[j] = k_{\text{raw}} \cdot 2704^{-1} \pmod{q} = k_{\text{raw}} \cdot 767 \pmod{3329},$$

since $2704 \times 767 \equiv 1 \pmod{3329}$. After this correction all 64 validation coefficients match exactly (**100% accuracy**).

2 Code Documentation

2.1 File Overview

```
python3 cs23b046.py polynomials.csv power_traces.csv
```

Output: `cs23b046_key.txt` — 256 recovered coefficients as a Python list. Total wall time: ≈ 8.8 s.

2.2 Algorithm Summary

Step 1. Load data. Read `polynomials.csv` ($N \times 256$, int32) and `power_traces.csv` ($N \times 640$, float32).

Step 2. Auto-detect leakage samples. For each coefficient c_i , build the sparse projection $W[c_i] = \mathbf{S}_{c_i}^\top \tilde{P}$ (where $\mathbf{S}_{c_i} \in \{0,1\}^{N \times q}$ is the one-hot indicator matrix) and set $\text{peak}(c_i) = \arg \max_t \|W[c_i]_{:,t}\|_2$.

Step 3. LUT-CPA over shift sweep. For each $\delta \in \{-5, \dots, +5\}$, build `coeff_tr`. For each $s \in \{0, \dots, q-1\}$, compute $\text{score}[s] = |\text{coeff_tr} \mathbf{h}_s|$ and update per-coefficient best score and best hypothesis.

Step 4. Montgomery correction. $s[j] = (k_{\text{best}} \times 767) \bmod 3329$.

Step 5. Write output. Save `cs23b046_key.txt`.

2.3 Annotated Source Code

```

1 def detect_peaks(polys, traces):
2     N, T = traces.shape
3     C = polys.shape[1]
4     row_i = np.arange(N, dtype=np.int32)
5
6     # Column-normalise traces (zero mean, unit std per sample)
7     tr_c = (traces - traces.mean(axis=0)).astype(np.float32)
8     tr_cn = (tr_c / (tr_c.std(axis=0) + 1e-12)).astype(np.float32)
9
10    peaks = np.zeros(C, dtype=np.int32)
11    for ci in range(C):
12        # Sparse indicator: row n fires at column polys[n, ci]
13        sp = csr_matrix(
14            (np.ones(N, dtype=np.float32),
15             (row_i, polys[:, ci])),
16            shape=(N, Q))
17        W_ci = sp.T @ tr_cn # (Q, T) projection
18        col_norms = np.linalg.norm(W_ci, axis=0) # (T,)
19        peaks[ci] = int(np.argmax(col_norms))
20    return peaks

```

Listing 1: Peak auto-detection via sparse projection

```

1 def cpa_attack(polys, traces):
2     lut = np.array([bin(i).count('1') for i in range(Q)],
3                    dtype=np.float32)
4     v = np.arange(Q, dtype=np.int32)
5
6     peaks = detect_peaks(polys, traces) # Step 1: auto-detect
7
8     max_corr_g = np.zeros(C, dtype=np.float32)
9     best_s_g = np.zeros(C, dtype=np.int32)
10
11    for shift in range(-5, 6): # Step 2: shift sweep
12        coeff_tr = build_coeff_tr(polys, traces, peaks, shift)
13
14        max_corr = np.zeros(C, dtype=np.float32)
15        best_s = np.zeros(C, dtype=np.int32)
16

```

```

17     for s in range(Q):                                     # Step 3: iterate
18         hypotheses                                         # lut_perm[r] = HW(r * s mod q)
19         lut_perm     = lut[np.mod(v * np.int32(s), Q)]
20         # score all 256 coefficients in one dot product
21         abs_corr_s = np.abs(coeff_tr @ lut_perm)
22         improved    = abs_corr_s > max_corr
23         max_corr     = np.where(improved, abs_corr_s, max_corr)
24         best_s       = np.where(improved, s, best_s)
25
26         improved_g = max_corr > max_corr_g
27         max_corr_g = np.where(improved_g, max_corr, max_corr_g)
28         best_s_g    = np.where(improved_g, best_s, best_s_g)
29
30     # Step 4: Montgomery correction
31     return (best_s_g.astype(np.int64) * 767 % Q).astype(np.int32)

```

Listing 2: Lookup-table CPA with shift sweep

3 Question 5

3.1 5a

Figure 1 shows the evolution of the peak absolute correlation score for a fixed coefficient (coefficient 0) as the number of power traces increases. Two curves are plotted: the correct key hypothesis (green solid line) and the best incorrect key hypothesis (red dashed line).

As the number of traces increases, the correlation corresponding to the correct key stabilizes and remains consistently higher than that of any incorrect key. This behavior is expected from Correlation Power Analysis (CPA), where the correct key hypothesis aligns with the true leakage model (Hamming weight), leading to a stronger linear relationship with the measured power consumption.

In contrast, the correlation for the best incorrect key decreases and converges to a lower value. This is because incorrect key guesses produce intermediate values that are statistically independent of the actual leakage, causing their correlation with the power traces to diminish as more samples are averaged.

The separation between the correct and incorrect key correlations becomes clearly visible after a sufficient number of traces (approximately a few hundred in this case). This gap indicates successful key recovery, as the correct key can be reliably distinguished from all incorrect candidates.

3.2 5b

Figure 2 shows the mean peak correlation obtained using different intermediate values in the Montgomery multiplication, averaged across coefficients. The intermediates m_0, m_1, m_2 , and m_3 correspond to partial products within the computation, while the “full” model refers to the Hamming Weight of the complete product $HW(r \cdot s \bmod q)$.

The results indicate that all individual intermediates exhibit relatively low correlation values (below 0.07). Among them, m_0 shows the highest leakage, followed by m_1 and

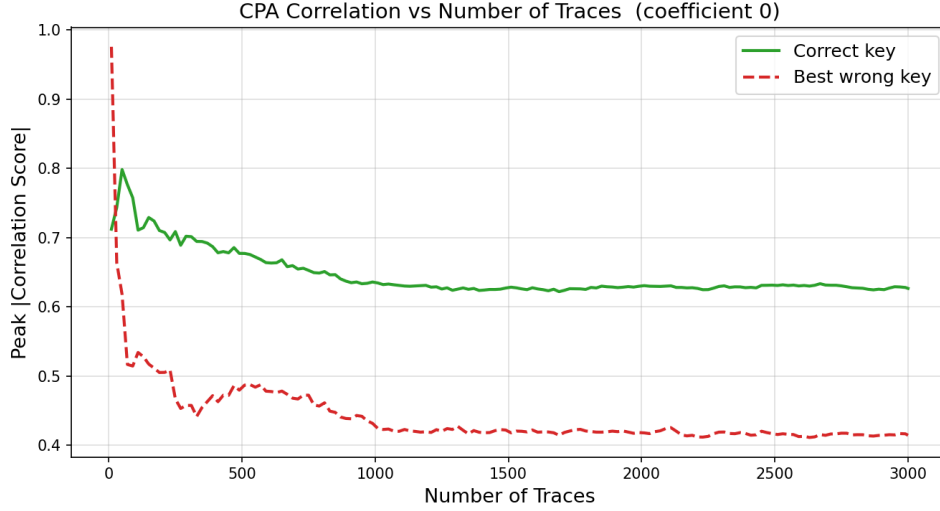


Figure 1: Peak correlation vs number of traces for coefficient 0. The correct key (green) stabilizes at a higher correlation, while the best incorrect key (red) decreases as more traces are used.

m_2 , while m_3 exhibits the weakest correlation. These low values suggest that individual sub-products leak only limited information about the secret key and are not sufficient for an effective CPA attack on their own.

In contrast, the full Hamming Weight model achieves a significantly higher mean correlation (approximately 0.63), clearly outperforming all intermediate-based models. This demonstrates that the dominant leakage arises from the final combined computation rather than from isolated intermediate steps.

The large gap between the full model and the intermediates highlights that leakage is not uniformly distributed across the computation. Instead, it is concentrated in later stages where the full product is formed, making it a more suitable target for CPA.

Overall, this figure emphasizes that while intermediate values may exhibit minor leakage, selecting a leakage model that captures the complete operation is crucial for achieving strong correlation and successful key recovery.

3.3 5c

Figure 3 compares the guessing entropy (GE) as a function of the number of traces for two leakage models: Hamming Weight (HW) and Hamming Distance (HD), averaged over the first 10 coefficients.

Guessing entropy represents the average rank of the correct key candidate among all hypotheses. A value of $GE = 1$ indicates perfect key recovery.

From the figure, the HW model rapidly converges to $GE = 1$ with a relatively small number of traces. This indicates that the Hamming Weight model accurately captures the underlying leakage behavior of the implementation, allowing the correct key to be consistently ranked first.

In contrast, the HD model performs significantly worse. Although the guessing entropy

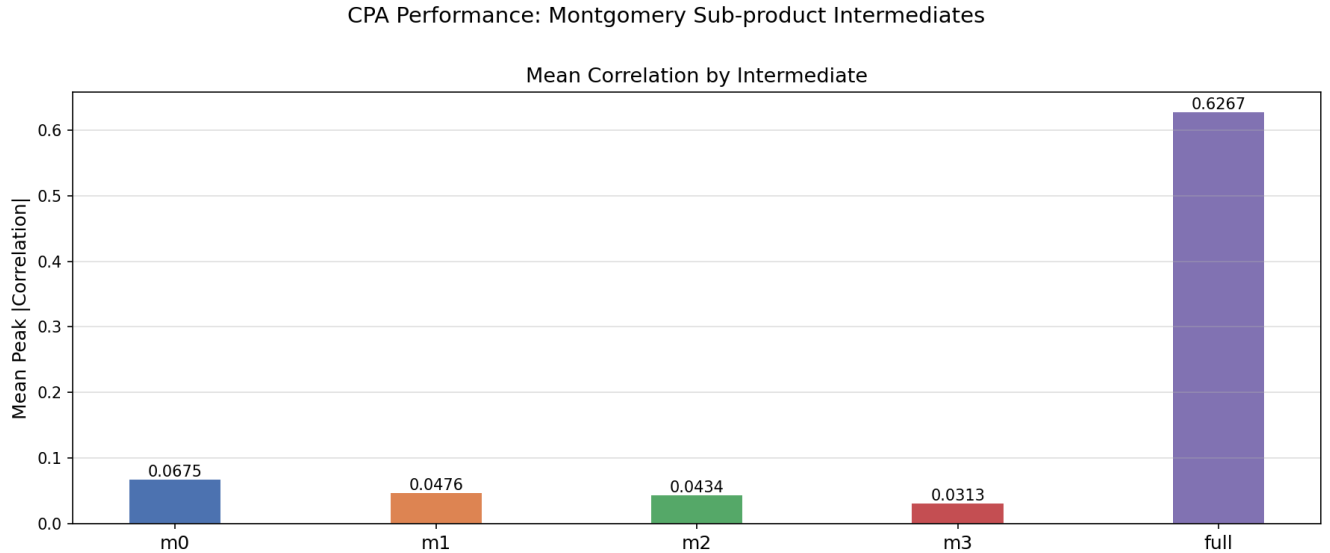


Figure 2: Mean peak correlation for different Montgomery sub-product intermediates compared to the full Hamming Weight model. Individual intermediates show weak leakage, while the full model achieves significantly higher correlation.

decreases initially as more traces are added, it stabilizes around a high value (approximately 1300). This indicates that the correct key is not reliably distinguished from incorrect hypotheses under the HD model.

Overall, the figure highlights the importance of selecting an appropriate leakage model: while the correct model (HW) enables efficient and reliable key recovery, an incorrect model (HD) leads to high guessing entropy and ineffective attacks.

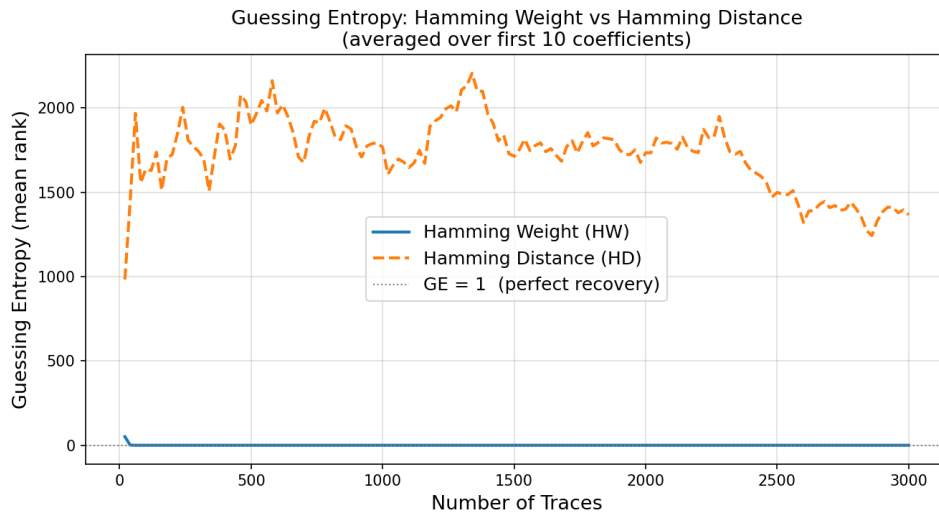


Figure 3: Guessing entropy vs number of traces for Hamming Weight (HW) and Hamming Distance (HD) models. HW quickly reaches $GE = 1$, while HD stabilizes around a high value, indicating poor key recovery performance.